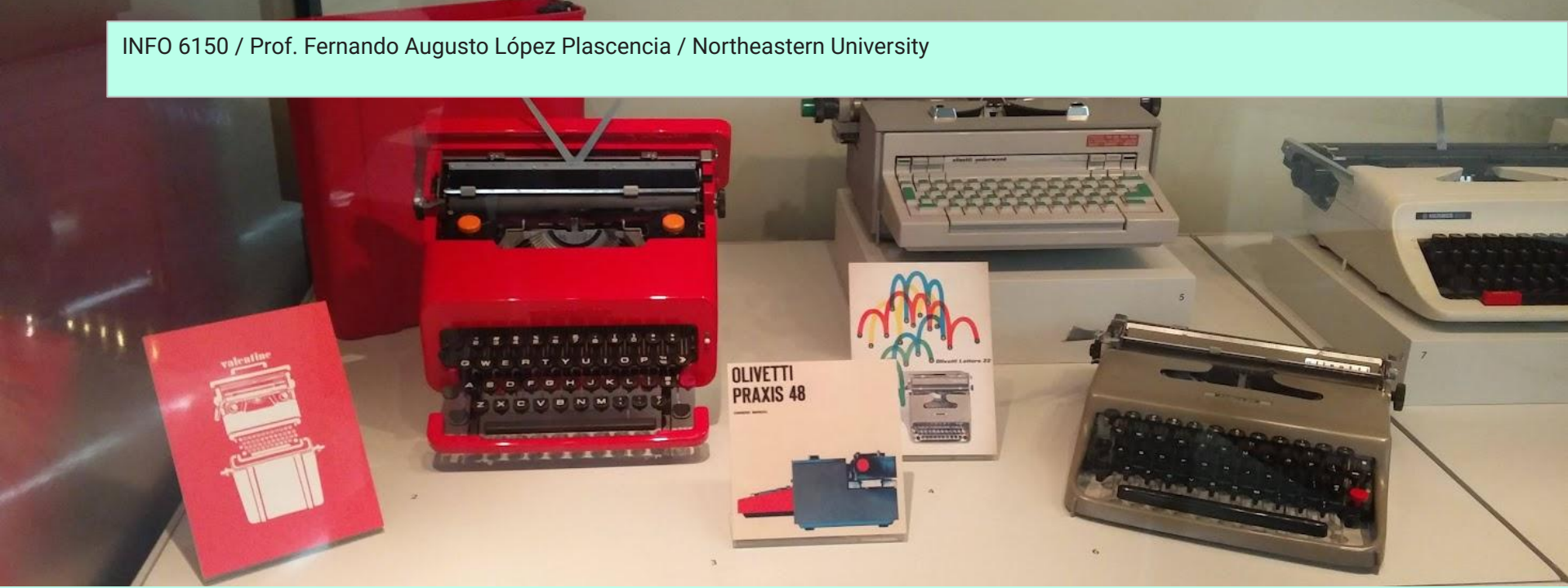


Week 10

TypeScript

In this lesson:

- Why types are a good idea
- Working with Typescript

Why types are a good idea

Why types are a good idea

At this point, you probably already realized something:

In Javascript, it’s easy to make mistakes by assuming a variable contains something different that it actually has.

Why types are a good idea

For example, you can do this in Javascript:

```
const { id } = user;
console.log(id);
```

But:

- What if the user object does not contain id?
- What if user is not even an object?

Why types are a good idea

You can check all of that manually with Javascript, and provide default values, but it's a hassle:

```
const id = user?.id ?? '';
console.log(id); // prints an empty string if there is no id
```

There are other, worse cases. Consider this:

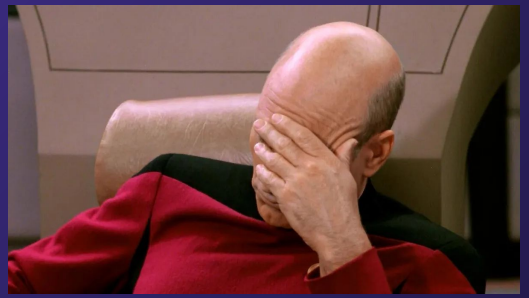
```
const sum = first + second;
console.log(sum);
```

Imagine that "first" and "second" are values received from input boxes. The user has inputted "1" in "first" and "2" in "second". Now, what is "sum" outputting?

Why types are a good idea

Well, that depends on whether you read those values correctly!

If you just got the “raw” value from the inputs, you’re getting *strings*. So, what you’re doing with `first + second` is actually concatenating the string “1” and the string “2”: sum now contains “12”.



Why (strong) types are a good idea

Types are a feature of many languages (“typed” languages).

Types are also in Javascript but they are “loose” types. A variable can change types depending on the context. For example:

```
const a = '1';
const b = 2;

const c = a + b;
console.log(c);
```

The output is “12”: Javascript dynamically changes the type of `b` from number to string, because `a` is a string, so it assumes we want to concatenate strings and not sum numbers.
This is not always what we want: often, when this happens, it’s a bug on our end.

Why (strong) types are a good idea

Other languages, such as Java, are *strong typed*: a variable or constant cannot change type. A number is always a number, never a string.

This is good because it makes for predictable code, not weird bugs.

Enter Typescript

Typescript is an extension of Javascript. It adds static type checking to Javascript:

- For objects and arrays, you can (actually you should) declare what type of data they contain. Then, it's more difficult to make mistakes.
  - For example: trying to insert a string in an array of numbers is no longer allowed.
- Because you're describing all types, you get better clues in your editor.
  - For example: if the shape of your object is wrong, you get the error while you're typing the code, not when you're running it.
- Autocomplete in the editor works better, so you can work faster.

All of these features become more important the bigger your project is, and the more complex your data and your objects are.

Enter Typescript

Typescript is a transpiled language. It won’t run by itself in the browser: it needs to be converted to standard Javascript first.

Enter Typescript

NOTE: Because Typescript builds on top of Javascript, it cannot really correct the original design errors of Javascript. So the following Typescript:

```
const a: number = 1;
const b: string = '2';

const c = a + b;
console.log(c);
```

still returns the string “12”. So, it’s not a silver bullet - but it’s still much better than nothing!

Getting started with Typescript

The best way to learn it is by checking the “Get Started” guides in https://www.typescriptlang.org/docs/ and practicing in the online playground at https://www.typescriptlang.org/play .

When you’re ready to use it in your projects, download and install it via npm, adding it as a dependency in your Node project with `npm install typescript --save-dev` . You may need to manually run the compiler (`tsc`) to compile your .ts files.

If starting a React project with Typescript, most frameworks already include ways to include Typescript. For a new Next.js project that uses Typescript, do `npx create-next-app@latest --typescript`.

Note: for this lesson we’ll mostly use the playground. The following slides are roughly based off this.

Typescript basics

In Typescript, we declare the types of our values, as follows:

```
// a number
const aNumber: number = 1;

// also a number; Typescript infers the type
const anotherNumber = 2;
```

Type inference means that Typescript will “guess” the type from the value.

Type inference works automatically for all primitive values, such as string, number, boolean, null, and undefined.

Typescript basics

We can also do the same for objects:

```
const book = {
  title: "How the states got their shapes",
  author: "Mark Stein",
  pages: 366,
};
```

infers that this object has title (a string), author (a string) and pages (a number). If we tried to do

```
book.pages = "100";
```

we get the error:

```
Type 'string' is not assignable to type 'number'.(2322)
```

Typescript basics

We can also declare the shape of the object. We can do it in a number of ways: with interfaces, classes, and types.

Interfaces are probably the simpler of the three. This is the one you will be using the most.

```
interface Book {
  title: string;
  author: string;
  pages: number;
}
```

To create an interface, use the interface keyword. Separate the members with a semicolon (;).

Typescript basics

Example:

```
interface Book {
  title: string;
  author: string;
  pages: number;
}
const anotherBook: Book = {
  title: "Kabuki",
  author: "Yukiko Kanazawa",
  pages: 12,
}
```

While we type our `anotherBook`, the editor will remind us that the shape of the object needs to be what we declared, and will nag us if we don't do it that way!

Typescript basics

We can also use classes. Classes come from Javascript, but they change slightly in Typescript:

```
class Book {
  name: string;
  author: string;
  pages: number;

  constructor(name: string, author: string, pages: number) {
    this.name = name;
    this.author = author;
    this.pages = pages;
  }
}

const book: Book = new Book("Clean Code", "Rpbert C. Martin", 512);
```

Typescript basics

We can also use classes. Classes come from Javascript, but they change slightly in Typescript:

```
class Book {
  name: string;
  author: string;
  pages: number;

  constructor(name: string, author: string, pages: number) {
    this.name = name;
    this.author = author;
    this.pages = pages;
  }
}

const book: Book = new Book("Clean Code", "Rpbert C. Martin", 512);
```

Typescript adds this.

Typescript basics

Finally, we can use types. Types are similar to interfaces, but the syntax changes:

```
type Book = {
  name: string;
  author: string;
  pages: number;
};

const book: Book = {
  name: "Success at Scale",
  author: "Addy Osmani",
  pages: 436,
};
```

Typescript basics

Finally, we can use types. Types are similar to interfaces, but the syntax changes:

```
type Book = {
  name: string;
  author: string;
  pages: number;
};

const book: Book = {
  name: "Success at Scale",
  author: "Addy Osmani",
  pages: 436,
};
```

We use =.
This will become relevant later

Typescript basics

For arrays, you declare them like this:

```
const books: Book[] = [
  {
    name: "Something",
    ...
```

Or of course you can start with an empty array: `const books: Book[] = [];`

in which case, Typescript will make sure you only introduce `Book` objects in there.

Typescript basics

Annotating functions: This is a very useful feature. You can specify:

- What are the types of the parameters of a function, like this:

```
function addBook(book: Book) {
  ...
}
```

- What value type the function returns, like this:

```
function getNextBook(): Book {
  ...
}
```

Typescript basics

This also works with arrow functions.

- What are the types of the parameters of a function:

```
function addBook(book: Book) {
  ...
}
```

- What value type the function returns, like this:

```
const getNextBook = (): Book => {
  ...
}
```

Typescript basics

What if a parameter is optional?

A parameter is optional if it can be omitted. In these cases, its value will undefined.

To mark a parameter as optional, we use a “?” at the end of its name:

```
function printName(obj: { first: string; last?: string }) {
  // ...
}
// Both OK
printName({ first: "Bob" });
printName({ first: "Alice", last: "Alisson" });
```

Type manipulation

Sometimes, a variable needs to be able to hold more than one type of value. For example, many functions return an object if successful, or `null` if not. Consider this example:

```
const divide = (num1, num2) => {
  if (num1 === 0 && num2 === 0) {
    return null;
  }
  return num1 / num2;
}
```

This function returns either a number or null.

Type manipulation

To represent this, we do:

```
const divisionResult: number | null = divide(1, 0);
```

The `number | null` is a type union.

Type manipulation

Effectively, the previous function in Typescript would be:

```
const divide = (num1: number, num2: number): number | null => {
  if (num1 === 0 && num2 === 0) {
    return null;
  }
  return num1 / num2;
}
```

Type manipulation

Effectively, the previous function in Typescript would be:

```
const divide = (num1: number, num2: number): number | null => {
  if (num1 === 0 && num2 === 0) {
    return null;
  }
  return num1 / num2;
}
```

The two arguments need to be numbers.

Type manipulation

Effectively, the previous function in Typescript would be:

```
const divide = (num1: number, num2: number): number | null => {
  if (num1 === 0 && num2 === 0) {
    return null;
  }
  return num1 / num2;
}
```

The return value can be either a number or null.

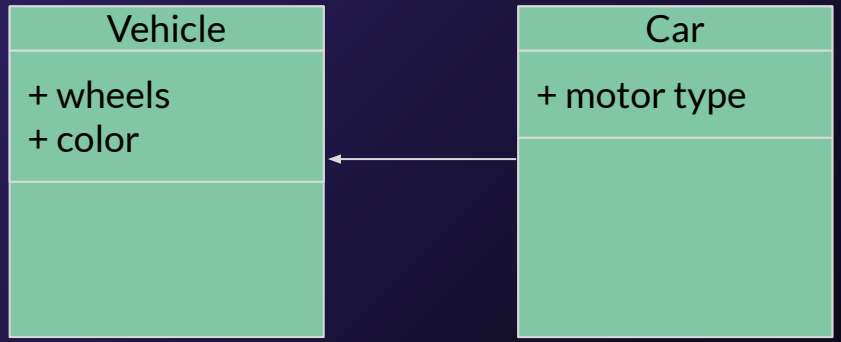
Type manipulation

Another possible thing to do is to extend it - that is, to create a new type based on another. This is useful to represent *things that are a kind of other thing*.

Take, for example:

```
type Vehicle = {
  wheels: number;
  color: string;
}

type Car = Vehicle & {
  motorType: string;
}
```



Type manipulation

So we could do:

```
type Vehicle = {
  wheels: number;
  color: string;
}

type Car = Vehicle & {
  motorType: string;
}

const newCar: Car = {
  wheels: 4,
  color: "#ffc",
  motorType: "electric",
};
```

Type manipulation

Most things that we can do with types, we can also do with interfaces.

For example, we can also base interfaces on other interfaces. This is called an “extension”.

The same example, with interfaces:

```
interface Vehicle {
  wheels: number;
  color: string;
}

interface Car extends Vehicle {
  motorType: string;
}

const newCar: Car = {
  wheels: 4,
  color: "#ffc",
  motorType: "electric",
};
```

The any type

There are two special types in Typescript that we'll talk about.

```
const returnSomething = (): any => { ... }
```

The first is any. You will use this to disable type

```
const a: number = returnSomething(); // ok
const b: string = returnSomething(); // also ok
```

With this, you're effectively undoing all Typescript benefits, so it's not recommended to use - except:

- As a temporary escape valve, if you're stuck with what types to use;
- As a temporary resource while you convert Javascript code to Typescript, and don't (or can't) do all of that at once.

Type assertion

Sometimes we know more than Typescript about what data we'll get from a function. Consider this:

```
interface Vehicle {
  wheels: number;
  color: string;
}
interface Car extends Vehicle {
  motorType: string;
}

const getLastVehicle = (): Vehicle => { ... }

const myCar = getLastVehicle();
```

Type assertion

If we know that getMyCar is actually a Car instance (a kind of Vehicle), we can do:

```
const getLastVehicle = (): Vehicle => { ... }
```

```
const myCar = getLastVehicle() as Car;
```

This allows us to do:

```
console.log(myCar.motorType);
```

which we wouldn’t be able to do if myCar was just a Vehicle.

Type assertion

The same assertion can be written as:

```
const myCar = <Car>getLastVehicle();
```

(Note: this syntax cannot be used in .tsx files - that is, in React files).

Type assertion

If Typescript detects that the assertion is not valid, you will get an error:

```
const x = "hello" as number;
```

```
Conversion of type 'string' to type 'number' may be a mistake because neither type sufficiently overlaps with
the other. If this was intentional, convert the expression to 'unknown' first.(2352)
```

Type assertion

If Typescript detects that the assertion is not valid, you will get an error:

```
const x = "hello" as number;
```

```
Conversion of type 'string' to type 'number' may be a mistake because neither type sufficiently overlaps with
the other. If this was intentional, convert the expression to 'unknown' first.(2352)
```

If you *know* the assertion should be valid you can do:

```
const x = "hello" as unknown as number;
```

(please don’t do this unless you *know* it will be okay… in this case it would not!)

Type assertion

If you *know* the assertion should be valid you can do:

```
const x = "hello" as unknown as number;
```

(please don't do this unless you *know* it will be okay… in this case it would not!)

Note, this uses another special type: unknown. This type means "we don't know the type and/or we don't care about the type". Unlike any values, unknown values cannot be used unless they're asserted first into a concrete type.

What is this value’s type?

Sometimes we don’t know what is the type of a value, and we need to know it. typeof allows us to know that. This is available even in Javascript:

```
const a = "text";
console.log(typeof a); // types “string”
```

What is this value's type?

In many cases, inputs can have different types. Consider this case:

```
const printTime = (now: string | Date): string => {
 // if now is a string, we return “now”; if it’s a Date, convert it to string first
}
```

How can we achieve this?

(By the way: `Date` is the Javascript representation for dates and times.)

What is this value's type?

We can do something like this:

```
const printTime = (now: string | Date): string => {
  if (typeof now === "string") {
    return now;
  }
  // at this point we know “now” is a Date
  return now.toDateString();
}
```

What is this object's type? Type guards

Now, imagine the following scenario:

```
interface Food { flavor: string };
interface Drink { base: "water" | "oil" };

const consume = (thing: Food | Drink): string => {
  // we cannot use typeof here; in any case, typeof thing === "object"
}
```

What is this object's type? Type guards

To solve this we create functions that tell us if an object is of a type. These are called "type guards".

```
interface Food { flavor: string };
interface Drink { base: "water" | "oil" };

const isFood = (item: any): item is Food => {
  return item && typeof item.flavor === "string";
}

const consume = (thing: Food | Drink): string => {
  if (isFood(thing)) {
    return "Eating";
  }
  // thing is a Drink
  return "Drinking";
}
```

What is this object's type? Type guards

To solve this we create functions that tell us if an object is of a type. These are called "type guards".

```
interface Food { flavor: string };
interface Drink { base: "water" | "oil" };

const isFood = (item: any): item is Food => {
  return item && typeof item.flavor === "string";
}

const consume = (thing: Food | Drink): string => {
  if (isFood(thing)) {
    return "Eating";
  }
  // thing is a Drink
  return "Drinking";
}
```

Means: the function returns true if "item" is has the type or interface Food.

Narrowing types

Some functions are made to return more than one type of data. Indeed, in software engineering we want to create DRY code (don’t repeat yourself), and for this we make functions that can be reused - and this sometimes means the function needs to return data of many types.

When dealing with the DOM and with React, you will find many of these functions. For example:

```
getElementById(elementId: string): HTMLElement | null;
```

which can return an HTMLElement object, or null.

Narrowing types

But what is an HTMLElement? The Mozilla docs tell us that

> The `HTMLElement` interface represents any HTML element.

It also says that HTMLElement *is a kind of* Element (it's its parent class), which itself is *a kind of* Node.

Also, HTMLElement has many children classes - that is, there are many kinds of HTMLElement.

For example, there is HTMLButtonElement, a class that represents a <button/>. It is a children of HTMLElement.

Narrowing types

Now consider this example, where we want to disable a button using TS:

```
const buttonNode = document.getElementById("#some-button");
buttonNode.disabled = true;
```

While this is not marked as an error, this code *can cause* an error. Even if buttonNode is found, it may not be a button, or any HTML node that does not contain a “disabled” attribute.

- Out of the many chlidren of HTMLElement, not all of them have a “disabled” attribute.

Also, in this case, the editor will not help us when writing the code.

Narrowing types

So, it's just better to specify the kind of HTMLElement that we expect:

```
const buttonNode = document.getElementById<HTMLButtonElement>("#some-button");
buttonNode.disabled = true;
```

In this case, TypeScript knows we specifically expect a button, and "disabled" will appear in the text prediction in the editor.

This is a kind of type narrowing. Other kinds we've already done (such as type guards). These practices are what helps your code in having less mistakes.

Using Typescript in real life

To use any Javascript package in your Typescript application, you will need the type declaration for that package. This is an additional file that is either

- included with the package (this is usually the case)
- or you have to install it separately by looking for it in the definitelytyped repo.
  - The repo includes instructions on how to look for them and how to install them.
  - At this point you should rarely need to do this anyway.

You can still use JS packages without type definitions, but everything you use with them will have the any type.

Using Typescript in real life

- DOM Manipulation: check the tutorial here for an exploration of how to do it.
- Typescript with React:
  - First check out this tutorial from the React folks. It includes the most common hooks.
  - Then check the JSX page in the Typescript handbook if you need more details.